BloodBond

Smart Blood Donation Network — Backend API

A .NET 9 Web API platform that connects blood donors with patients in need, intelligently matching compatible blood types across blood banks.

Project Type: **Academic Course Delivery**

Architecture: **3-Layer (DAL / BLL / UI)**

Database: **SQL Server (Code-First with EF Core 9)**

Authentication: **ASP.NET Core Identity + JWT Bearer**

Payment: **Stripe Checkout**

Document version: **1.0 — August 2026**

Generated: 2026-08-08

Table of Contents

1. Project Overview

BloodBond is a backend API for a smart blood donation network. It enables users to register, request blood, schedule donations, manage blood bank inventory, and make monetary contributions to support blood bank operations.

Primary users:

• **Donor** — gives blood or money

• **Requester** — patient/family who needs blood

• **Blood Bank Manager** — manages a bank's inventory and donations

• **Admin** — approves banks, manages users, monitors system

Implemented in this delivery (50%+):

• Identity & Authentication (with JWT)

• Blood Bank verification workflow

• Inventory tracking + low-stock alerts

• Smart matching of compatible donors (by blood type + city)

• Emergency notifications for critical requests

• Pre-donation eligibility screening

• Full donation lifecycle (schedule → approve → complete)

• Voluntary monetary donations via Stripe Checkout

• Admin user management (create, block, change role)

2. Architecture & Project Structure

The project follows a clean **3-Layer Architecture** pattern that keeps concerns separated, makes the system testable, and lets new modules plug in without rewiring the rest.

Project layout:

```
BloodBond/
■■■ BloodBond.sln
■■■ BloodBond.DAL/          # Data Access Layer
■   ■■■ Models/             # 10+ entity classes
■   ■■■ Data/               # ApplicationDbContext
■   ■■■ Repository/         # Generic + 7 specific repositories
■   ■■■ DTO/                # 30+ request/response DTOs
■   ■■■ Enums/              # BloodType, UrgencyLevel, etc.
■   ■■■ utils/              # BloodCompatibility, RoleSeedData
■   ■■■ Migrations/         # EF Core migrations
■■■ BloodBond.BLL/          # Business Logic Layer
■   ■■■ Service/            # 8 services (Auth, Bank, Request, etc.)
■   ■■■ Mapping/            # MapsterConfig
■■■ BloodBond/              # Presentation Layer
    ■■■ Controllers/        # 8 REST controllers
    ■■■ Extensinos/         # DI / service registration
    ■■■ Middleware/         # GlobalExceptionHandling
    ■■■ Program.cs
    ■■■ appsettings.json
```

3. Tech Stack

Category	Technology
Framework	ASP.NET Core 9 Web API
ORM	Entity Framework Core 9 (Code-First)
Database	SQL Server (LocalDB / Express / full)
Auth	ASP.NET Core Identity + JWT Bearer
Mapping	Mapster
Payments	Stripe.net (Stripe Checkout)
Validation	Data Annotations
Patterns	Repository, Generic Repository, DI, Seeding

4. Database Schema (Code-First)

All tables are generated by EF Core from C# classes. There is no manual SQL to maintain.

Tables created (in this delivery):

Table	Purpose
Users	Application users (extends IdentityUser)
Roles	Admin / User / BloodBankManager
UserRoles	Identity join table
BloodBanks	Registered blood banks (Pending/Verified/Rejected)
BloodInventories	Stock per blood bank per blood type
BloodRequests	Blood requests by requesters
Donations	Donation appointments
EligibilityAnswers	Pre-donation screening answers
Notifications	User notifications (info/emergency)
MonetaryDonations	Stripe monetary donations
__EFMigrationsHistory	EF Core tracking

5. Implemented Modules (Slices)

#	Slice	What's in it
1	Identity & Auth	ApplicationUser, Roles, JWT, AccountController, AdminController
2	BloodBank & Inventory	BloodBank CRUD, Approve/Reject, Inventory, Low-stock
3	BloodRequest & Matching	Create/Cancel requests, Smart matching, Notifications
4	Eligibility & Donation	Eligibility screening, Schedule/Approve/Reject/Complete donations
5	Stripe Monetary Donations	Checkout Session, list donations, totals
6	Admin User Management	Create users, block/unblock, change role, change password

#	Slice	What's in it
1	Identity & Auth	ApplicationUser, Roles, JWT, AccountController, AdminController
2	BloodBank & Inventory	BloodBank CRUD, Approve/Reject, Inventory, Low-stock
3	BloodRequest & Matching	Create/Cancel requests, Smart matching, Notifications
4	Eligibility & Donation	Eligibility screening, Schedule/Approve/Reject/Complete donations
5	Stripe Monetary Donations	Checkout Session, list donations, totals
6	Admin User Management	Create users, block/unblock, change role, change password

6. User Flows Covered

From the official Business Model document, the following flows are end-to-end functional:

Donor flow (9 steps):

1. Register an account → POST /api/account/register
2. Log in → POST /api/account/login (returns JWT)
3. Complete profile (blood type, city) → user record
4. Browse compatible requests → GET /api/bloodrequests/active?city=...
5. Schedule a donation at a verified blood bank → POST /api/donations
6. Complete eligibility screening → POST /api/eligibility
7. Blood bank manager marks donation complete → PATCH /api/donations/{id}/complete
8. Donor history updates, points awarded (auto)
9. Optionally donate money via Stripe → POST /api/monetarydonations/create-intent

Requester flow (5 steps):

1. Register + login (same Auth as donor)
2. Create a blood request with type, units, urgency, city → POST /api/bloodrequests
3. System notifies compatible donors in the same city (smart matching)
4. Track the request status → GET /api/bloodrequests/mine
5. Cancel or mark fulfilled → PATCH /api/bloodrequests/{id}/cancel or /fulfill

Blood bank manager flow (7 steps):

1. Register a blood bank (auto-promoted to BloodBankManager role) → POST /api/bloodbanks
2. Wait for Admin verification → status = Pending
3. Manage inventory per blood type → PUT /api/bloodbanks/{id}/inventory
4. Admin approves → PATCH /api/bloodbanks/{id}/approve
5. Review scheduled donation appointments → GET /api/donations/by-bank/{id}
6. Approve/Reject/Complete → PATCH /api/donations/{id}/approve|reject|complete
7. View donations for their bank → GET /api/monetarydonations/by-bank/{id}

Admin flow (subset):

1. Approve / reject blood bank registrations
2. Manage users: create, change role, block/unblock
3. Change own password

7. REST API Endpoints (Reference)

Base URL: **https://localhost:7000**

All endpoints return JSON. Authenticated endpoints require a JWT Bearer token in the Authorization header.

Method	URL	Auth	Description
POST	/api/account/register	Anonymous	Register a new user (default role: User)
POST	/api/account/login	Anonymous	Authenticate and receive a JWT token
POST	/api/account/forgot-password	Anonymous	Request a password reset email
POST	/api/account/reset-password	Anonymous	Reset the password using a token
GET	/api/account/me	Auth	Get the currently authenticated user
POST	/api/admin/register-first	SecretKey	Bootstrap the FIRST admin (only if no admin exists)
POST	/api/admin/create	Admin	Create a new user with a specific role
POST	/api/admin/change-password	Admin	Change the current admin's own password
GET	/api/admin/users	Admin	List all users
GET	/api/admin/users/{id}	Admin	Get a specific user
PATCH	/api/admin/users/{id}/block	Admin	Block a user
PATCH	/api/admin/users/{id}/unblock	Admin	Unblock a user
PATCH	/api/admin/users/{id}/role	Admin	Change a user's role
GET	/api/bloodbanks	Anonymous	List all blood banks
GET	/api/bloodbanks/verified	Anonymous	List verified blood banks only
GET	/api/bloodbanks/{id}	Anonymous	Get a blood bank by id
GET	/api/bloodbanks/low-stock	Anonymous	List inventory items with units < 5
POST	/api/bloodbanks	Auth	Create a blood bank (auto-promotes creator to manager)
GET	/api/bloodbanks/mine	Manager	Get the bank managed by the current user
PUT	/api/bloodbanks/{id}	Manager	Update the bank you manage
PUT	/api/bloodbanks/{id}/inventory	Manager	Replace the entire inventory list
PATCH	/api/bloodbanks/{id}/approve	Admin	Approve a pending bank
PATCH	/api/bloodbanks/{id}/reject	Admin	Reject a pending bank
POST	/api/bloodrequests	Auth	Create a blood request and notify compatible donors
GET	/api/bloodrequests/mine	Auth	List my own requests
GET	/api/bloodrequests/active?city=...	Anonymous	Active requests in a city (smart matching prep)
GET	/api/bloodrequests/{id}	Auth	Get a specific request
PATCH	/api/bloodrequests/{id}/cancel	Auth	Cancel a request you created
PATCH	/api/bloodrequests/{id}/fulfill	Manager/Admin	Mark request as fulfilled
POST	/api/bloodrequests/{id}/notify	Manager/Admin	Re-notify compatible donors
POST	/api/donations	Auth	Schedule a donation (requires passed eligibility)
GET	/api/donations/mine	Auth	List my donations
GET	/api/donations/{id}	Auth	Get a specific donation
PATCH	/api/donations/{id}/cancel	Auth	Cancel a scheduled donation
GET	/api/donations/by-bank/{bankId}	Manager/Admin	List donations for a bank
PATCH	/api/donations/{id}/approve	Manager/Admin	Approve a scheduled donation

Method	URL	Auth	Description
PATCH	/api/donations/{id}/reject	Manager/Admin	Reject a scheduled donation
PATCH	/api/donations/{id}/complete	Manager/Admin	Mark complete → updates inventory + donor points
POST	/api/eligibility	Auth	Submit pre-donation screening answers
GET	/api/eligibility/latest	Auth	Get my most recent eligibility answer
POST	/api/monetarydonations/create-intent	Auth	Create a Stripe Checkout Session (returns checkoutUrl)
POST	/api/monetarydonations/webhook	Anonymous	Stripe webhook for payment status updates
POST	/api/monetarydonations/confirm	Anonymous	Manual confirm (for testing without webhook)
GET	/api/monetarydonations/mine	Auth	List my monetary donations
GET	/api/monetarydonations/by-bank/{bankId}	Manager/Admin	Donations for a bank
GET	/api/monetarydonations/total/mine	Auth	Total money I've donated
GET	/api/monetarydonations/success	Anonymous	Stripe Checkout success redirect
GET	/api/monetarydonations/cancel	Anonymous	Stripe Checkout cancel redirect

8. Setup & Configuration

Prerequisites:

• .NET 9 SDK

• SQL Server (LocalDB / Express / full instance)

• Visual Studio 2022 / VS Code

Steps:

1. **Clone the repository** (or open the project folder).

2. **Configure the connection string** in appsettings.json:

```
"ConnectionStrings": {
  "DefaultConnection": "Data Source=.;Initial Catalog=BloodBondDb;Integrated Security=True;Trust Server Certifi
}
```

3. **Apply database migrations**:

```
dotnet ef database update --project BloodBond.DAL --startup-project BloodBond
```

4. **Run the application**:

```
dotnet run --project BloodBond
```

5. The application starts on **https://localhost:7000** (Swagger available there).

6. **Bootstrap the first admin** by sending a POST request (only if no admin exists yet):

```
POST /api/admin/register-first
Content-Type: application/json

{
  "secretKey": "CHANGE_THIS_TO_A_LONG_RANDOM_STRING_BEFORE_PRODUCTION",
  "fullName": "Your Name",
  "email": "admin@yourcompany.com",
  "password": "SecurePassword@123"
}
```

7. **Log in** as the new admin and start using the API.

Note: The default admin account (admin@bloodbond.com / Admin@123456) was created during development and is stored in the database. You can change it via the admin endpoints.

9. Postman Testing Guide (Step by Step)

This section walks through every endpoint with concrete Postman examples. Each step has a screenshot-ready request shape.

Step 0 — Set up Postman environment

Create a Postman environment with these variables so you don't have to repeat the URL everywhere:

```
baseUrl = https://localhost:7000
adminToken  = (filled after admin login)
donorToken  = (filled after donor login)
requesterToken = (filled after requester login)
bankId = (filled after creating a bank)
```

Step 1 — Bootstrap the first admin (only works if no admin exists)

```
POST {{baseUrl}}/api/admin/register-first
Content-Type: application/json

{
  "secretKey": "CHANGE_THIS_TO_A_LONG_RANDOM_STRING_BEFORE_PRODUCTION",
  "fullName": "My Admin",
  "email": "myadmin@example.com",
  "password": "MyAdmin@123"
}
```

Step 2 — Log in as admin (capture the token)

```
POST {{baseUrl}}/api/account/login
Content-Type: application/json

{
  "email": "admin@bloodbond.com",
  "password": "Admin@123456"
}

// → copy the value of "token" into {{adminToken}}
```

✓ Tip: copy the value of "token" from the response into a Postman environment variable called **adminToken** for reuse.

Expected response:

```
{
  "userId": "4b61c196-9d91-4de4-a519-86552a61c057",
  "email": "admin@bloodbond.com",
  "fullName": "System Admin",
  "roles": ["Admin"],
  "token": "eyJhbGciOi...",
  "expiresAt": "2026-08-14T08:34:20Z"
}
```

Step 3 — Register a donor

```
POST {{baseUrl}}/api/account/register
Content-Type: application/json

{
  "fullName": "Test Donor",
  "email": "donor.test@example.com",
  "password": "Donor@1234",
  "confirmPassword": "Donor@1234"
}
```

Step 4 — Log in as the donor

```
POST {{baseUrl}}/api/account/login
Content-Type: application/json
```

```
{
  "email": "donor.test@example.com",
  "password": "Donor@1234"
}

// → copy the value of "token" into {{donorToken}}
```

Step 5 — Create a blood bank (donor becomes a manager)

```
POST {{baseUrl}}/api/bloodbanks
Authorization: Bearer {{donorToken}}
Content-Type: application/json

{
  "name": "Central Blood Bank",
  "cityAddress": "123 Main St, Ramallah",
  "latitude": 31.9038,
  "longitude": 35.2030,
  "contactPhone": "+970599123456"
}

// → copy the value of "id" into {{bankId}}
```

Step 6 — Re-login the donor to pick up the BloodBankManager role

After creating the bank, repeat Step 4 so the JWT contains the new role.

Step 7 — Set the bank's inventory

```
PUT {{baseUrl}}/api/bloodbanks/{{bankId}}/inventory
Authorization: Bearer {{donorToken}}
Content-Type: application/json

[
  { "bloodType": 0, "unitsAvailable": 10 },
  { "bloodType": 1, "unitsAvailable": 3 },
  { "bloodType": 6, "unitsAvailable": 8 }
]
```

Step 8 — Approve the bank (admin)

```
PATCH {{baseUrl}}/api/bloodbanks/{{bankId}}/approve
Authorization: Bearer {{adminToken}}
```

Step 9 — Eligibility screening (donor)

```
POST {{baseUrl}}/api/eligibility
Authorization: Bearer {{donorToken}}
Content-Type: application/json

{
  "weight": 75,
  "age": 28,
  "hasChronicDisease": false
}
```

Step 10 — Schedule a donation

```
POST {{baseUrl}}/api/donations
Authorization: Bearer {{donorToken}}
Content-Type: application/json

{
  "bloodBankId": 1,
  "scheduledDate": "2026-08-15T10:00:00",
  "notes": "Regular donation"
}
```

Step 11 — Approve + complete the donation (manager = donor)

```
PATCH {{baseUrl}}/api/donations/1/approve
Authorization: Bearer {{donorToken}}
```

```
PATCH {{baseUrl}}/api/donations/1/complete
Authorization: Bearer {{donorToken}}
Content-Type: application/json

{ "unitsDonated": 1, "notes": "Successful donation" }
```

Step 12 — Register a requester and create a blood request

```
POST {{baseUrl}}/api/account/register
{ "fullName": "Test Requester", "email": "req@example.com",
  "password": "Req@1234", "confirmPassword": "Req@1234" }

POST {{baseUrl}}/api/bloodrequests
Authorization: Bearer {{requesterToken}}
Content-Type: application/json

{
  "bloodType": 0,
  "unitsNeeded": 2,
  "urgencyLevel": 2,
  "city": "Ramallah"
}
```

Step 13 — Money donation via Stripe

```
POST {{baseUrl}}/api/monetarydonations/create-intent
Authorization: Bearer {{donorToken}}
Content-Type: application/json

{ "amount": 25.00, "currency": "usd" }

// → copy the value of "checkoutUrl" and open it in a browser
// Pay with test card 4242 4242 4242 4242
```

10. Test Results — Verified Endpoints

All phases were run end-to-end against the running API and produced the expected results.

Phase	Module	Endpoints tested	Status
1	Auth	register / login / me	■ PASSED
2	BloodBank + Inventory	create / mine / setInventory / low-stock / approve	■ PASSED
3	Eligibility + Donation	check / schedule / approve / complete / mine / by-ban	■ PASSED
4	BloodRequest + Matching	create / mine / active?city / notify / cancel	■ PASSED
5	Stripe Monetary Donations	create-intent / mine / total	■ PASSED

11. Bugs Found & Fixed

During testing, two real bugs were found and fixed:

Bug #1 — EF Core could not translate BloodCompatibility.CanDonateTo

Symptom: POST /api/bloodrequests returned 500 after the donor notification step.

Cause: LINQ tried to translate the static helper into SQL, which is not supported.

Fix: split the query into two steps — SQL filter (city, has blood type, not blocked) followed by an in-memory compatibility check.

Also wrapped the notification call in try/catch so a notification failure cannot break the request creation.

Bug #2 — Hidden 500 on /api/bloodrequests from unhandled exception

Symptom: response was 500 but the log showed nothing for the request itself.

Fix: now logged in the catch block and the request creation always succeeds even if notification dispatch fails.

12. Stripe Payment Flow

Monetary donations use **Stripe Checkout** — a hosted payment page. The flow is:

```
1. Donor calls POST /api/monetarydonations/create-intent
   → Backend creates a Stripe Checkout Session
   → Returns a checkoutUrl like
     https://checkout.stripe.com/c/pay/cs_test_xxx...

2. Frontend (or Postman) opens that URL in a browser

3. Donor pays with the test card
     Card: 4242 4242 4242 4242
     Exp:  any future date
     CVC:  any 3 digits

4. Stripe redirects back to
     GET /api/monetarydonations/success?session_id=...
   (or /cancel if the donor cancels)

5. The success endpoint marks the donation as Succeeded

6. (Optional) Stripe also POSTs the webhook to
     /api/monetarydonations/webhook
   which updates the donation in the background
   (requires a configured WebhookSecret)
```

Mock mode is automatic: if Stripe:SecretKey is left empty or contains a placeholder, the system returns a fake payment intent with isMock=true. This lets the rest of the flow be developed without a real Stripe account.

13. How to Extend (Next Steps)

The architecture makes it straightforward to add more modules without touching the existing ones. Typical follow-up deliveries could add:

• **Blood Drive Events** — managers create events, donors register and check in

• **Ratings** — donors rate the blood bank they visited

• **Badges & Gamification** — auto-award badges based on donation count / streaks

• **Real-time notifications** — SignalR push instead of DB-only notifications

• **Localization** — English + Arabic error messages and notifications

• **Admin analytics dashboard** — inventory levels, fulfillment rate, blood type distribution

• **Production hardening** — move secrets to environment variables, enable HTTPS-only, add rate limiting, add CI/CD

End of document

Generated automatically from the running project on 2026-08-08.